

What is a database?

A **database** is a place where your application stores data permanently.

For example, suppose you create a **Student Management System**.

You need to store:

	id	name	email	course
--	----	------	-------	--------

1	Rahul	rahul@gmail.com	Python
---	-------	-----------------	--------

2	Amit	amit@gmail.com	Flask
---	------	----------------	-------

Without a database, this information would disappear when your Flask application stops.

How Flask works with a database

User



Browser / Postman



Flask Application



Database Layer



MySQL / PostgreSQL / SQLite

POST /students



Flask receives student data



Validate data



Insert into database



Database saves student



Flask sends response

Which databases can Flask use?

You can use:

- SQLite
- MySQL
- PostgreSQL
- MongoDB

beginners

Flask + SQLite + SQLAlchemy

Later:

Flask + PostgreSQL + SQLAlchemy

What is SQLAlchemy?

This is very important.

SQLAlchemy is a Python SQL toolkit and ORM.

ORM = Object Relational Mapping

Instead of writing SQL for every operation

```
INSERT INTO students (name, email)
```

```
VALUES ('Rahul', 'rahul@gmail.com');
```

```
student = Student(  
    name="Rahul",  
    email="rahul@gmail.com"  
)
```

```
db.session.add(student)
```

```
db.session.commit()
```

So you can think:

Python Class



SQLAlchemy ORM



Database Table

For example:

```
class Student(db.Model):  
    id = db.Column(db.Integer, primary_key=True)  
    name = db.Column(db.String(100))  
    email = db.Column(db.String(100))
```

This represents a database table.

Database table vs Python class

This is one of the most important concepts.

Python:

```
class Student:
```

```
    id  
    name  
    email
```

Database:

students

id | name | email

1 | Rahul | rahul@gmail.com

2 | Amit | amit@gmail.com

SQLAlchemy connects these two.

Student class



SQLAlchemy



students table

Installing Flask database packages

Create your virtual environment and install:

```
pip install flask flask-sqlalchemy
```

```
pip show flask-sqlalchemy
```

Simple Flask + SQLite project

Project structure:

flask_database/

|

├─ app.py

└─ students.db

```
from flask import Flask

from flask_sqlalchemy import SQLAlchemy

app = Flask(__name__)

app.config["SQLALCHEMY_DATABASE_URI"] = "sqlite:///students.db"

db = SQLAlchemy(app)

class Student(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(100), nullable=False)
    email = db.Column(db.String(100), unique=True, nullable=False)

with app.app_context():
    db.create_all()

@app.route("/")
def home():
    return "Flask Database is Working"

if __name__ == "__main__":
    app.run(debug=True)
```

Flask-SQLAlchemy will create:

students.db

And inside it, you'll have a:

student

CRUD operations

When working with databases, you should learn **CRUD** first.

C → Create

R → Read

U → Update

D → Delete

These are the fundamental database operations.

Python:

```
student = Student(  
    name="Rahul",  
    email="rahul@gmail.com"  
)
```

```
db.session.add(student)
```

```
db.session.commit()
```

Think of it as:

Create Student

↓

Add to database session

↓

Commit

↓

Data permanently saved

Read data:

```
@app.route("/students")
def get_students():

    students = Student.query.all()

    return {
        "students": [
            {
                "id": student.id,
                "name": student.name,
                "email": student.email
            }
            for student in students
        ]
    }
```

Update:

Suppose Rahul changes his email.

SQLite vs MySQL vs PostgreSQL

SQLite

Good for:

Learning

Small projects

Testing

Prototypes

Database is basically a file:

students.db

MySQL

Common for:

Web applications

Business applications

Production systems

PostgreSQL

Excellent for:

Large applications

Complex queries

Production APIs

Data-intensive applications

lunetron web services


```
from flask import Flask, request, jsonify
from flask_sqlalchemy import SQLAlchemy

app = Flask(__name__)

# Database configuration
app.config["SQLALCHEMY_DATABASE_URI"] = "sqlite:///students.db"
app.config["SQLALCHEMY_TRACK_MODIFICATIONS"] = False

db = SQLAlchemy(app)

# Model / Table
class Student(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(100), nullable=False)
    email = db.Column(db.String(100), unique=True, nullable=False)
    course = db.Column(db.String(100), nullable=False)

# Create database
with app.app_context():
    db.create_all()

# CREATE - POST
@app.route("/students", methods=["POST"])
def create_student():

    data = request.get_json()
```

```
student = Student(  
    name=data["name"],  
    email=data["email"],  
    course=data["course"]  
)  
  
db.session.add(student)  
db.session.commit()  
  
return jsonify({  
    "message": "Student created successfully",  
    "id": student.id  
}), 201
```

READ - GET ALL

```
@app.route("/students", methods=["GET"])  
def get_students():
```

```
    students = Student.query.all()
```

```
    result = []
```

```
    for student in students:
```

```
        result.append({  
            "id": student.id,  
            "name": student.name,  
            "email": student.email,  
            "course": student.course  
        })
```

```
return jsonify(result), 200
```

```
# READ - GET ONE
```

```
@app.route("/students/<int:id>", methods=["GET"])
```

```
def get_student(id):
```

```
    student = Student.query.get(id)
```

```
    if not student:
```

```
        return jsonify({
```

```
            "message": "Student not found"
```

```
        }), 404
```

```
    return jsonify({
```

```
        "id": student.id,
```

```
        "name": student.name,
```

```
        "email": student.email,
```

```
        "course": student.course
```

```
    }), 200
```

```
# UPDATE - PUT
```

```
@app.route("/students/<int:id>", methods=["PUT"])
```

```
def update_student(id):
```

```
    student = Student.query.get(id)
```

```
    if not student:
```

```
        return jsonify({
```

```
            "message": "Student not found"
```

```
}}, 404
```

```
data = request.get_json()
```

```
student.name = data["name"]
```

```
student.email = data["email"]
```

```
student.course = data["course"]
```

```
db.session.commit()
```

```
return jsonify({
```

```
    "message": "Student updated successfully"
```

```
}), 200
```

```
# DELETE - DELETE
```

```
@app.route("/students/<int:id>", methods=["DELETE"])
```

```
def delete_student(id):
```

```
    student = Student.query.get(id)
```

```
    if not student:
```

```
        return jsonify({
```

```
            "message": "Student not found"
```

```
        }), 404
```

```
db.session.delete(student)
```

```
db.session.commit()
```

```
return jsonify({
```

```
    "message": "Student deleted successfully"
```

```
}}, 200
```

```
if __name__ == "__main__":  
    app.run(debug=True)
```

lunetron web services